

Smart Greenhouse Sensor Simulation

1. Introduction

Modern greenhouses rely on networks of environmental sensors to maintain optimal growing conditions. These sensors continuously record temperature, humidity, soil moisture, and light levels across different zones of the greenhouse.

As part of the development team for a new Smart Greenhouse Monitoring System, you are responsible for building the data analysis module. Your program will load real sensor data, generate meaningful statistics, and present insights that help greenhouse operators make informed decisions.

This assignment consolidates your skills in pseudocode design, class modelling, procedural algorithms, Java implementation, data analysis, and reflective communication.

2. Unit Learning Outcomes Assessed

Upon successful completion of this assignment, you will be able to:

1. Identify appropriate primitive data types required for the translation of pseudocode algorithms into Java.
2. Design simple classes in pseudocode and implement them in Java in a Linux command line environment.
3. Design and implement structured procedural algorithms in pseudocode and Java.
4. Apply design and programming skills to implement known algorithms in real world applications.
5. Reflect on design choices and communicate decisions in a manner appropriate to the audience.

3. Scenario Background

The greenhouse is divided into multiple zones (e.g., Zone A, Zone B, Zone C). Each zone contains several sensors that record environmental conditions at regular intervals.

You have been provided with a CSV dataset containing thousands of sensor readings. Each row represents a single reading from a specific sensor at a specific time.

Your task is to design and implement a Java program that:

- loads the dataset
- stores readings as objects
- allows the user to explore the data through a menu system
- computes meaningful statistics
- writes all output to a log file
- reflects on your design choices

4. Provided Dataset

A CSV file named `greenhouse_readings.csv` is provided. Below is a Sample Dataset:

```
day,month,year,hour,minute,sensorID,sensorType,zone,value
12,3,2024,14,30,TMP332,temperature,ZoneA,27.4
12,3,2024,14,30,HMD221,humidity,ZoneB,61.2
12,3,2024,14,30,S0IL88,soilMoisture,ZoneA,42.0
12,3,2024,14,30,LGT901,light,ZoneC,880
```

Sensor Types and Safe Ranges

Sensor Type	Safe Range
temperature	18–30 °C
humidity	40–70 %
soilMoisture	30–60 %
light	300–1200 lux

Your program must use these safe ranges when calculating statistics.

5. Required Classes

The following structure is a requirement. If you wish to add more fields or more classes, that is acceptable. However, the structure listed below is mandatory.

5.1 Timestamp Class

Fields:

- dayOfMonth (int)
- monthOfYear (int)
- year (int)
- hour (int)
- minute (int)

5.2 SensorReading Class

Fields:

- sensorID (String)
- sensorType (String)
- zone (String)
- value (double)
- timestamp (Timestamp object)

Requirements

- Correct primitive types
- Composition (SensorReading contains a Timestamp)
- Constructors (all three discussed in this unit)
- Accessor methods (all discussed in this unit)
- Mutator methods (all discussed in this unit)
- Clear pseudocode design before implementation

6. Program Requirements

6.1 Data Loading

Your program must:

- automatically load the CSV file when the program starts
- parse each row into a SensorReading object
- store all objects in an array (not ArrayList)
- validate data (e.g., numeric values, valid sensor types)

6.2 Menu System

Your program must display the following main menu:

Welcome to the Smart Greenhouse Monitoring System.

Please select an option:

- 1. Statistics for the entire greenhouse**
- 2. Statistics by zone**
- 3. Statistics by sensor type**
- 4. Add data readings**
- 5. Delete data readings**
- 6. Exit program**

6.3 Submenus

Select a statistic:

- 1. Total number of readings**
- 2. Average value**
- 3. Minimum value**
- 4. Maximum value**
- 5. Number of readings outside safe range**
- 6. Percentage of readings outside safe range**
- 7. All statistics**

6.4 Output Requirements

- All results must be displayed clearly on screen.
- All results must also be written to a log file.
- After each action, the program must return to the main menu.

7. Required Calculations

Your program must compute:

- total number of readings
- average reading value
- minimum and maximum values
- number of readings outside safe range
- percentage outside safe range
- comparisons between zones or sensor types

8. Input Validation

Your program must validate:

- menu choices
- sensor type selections
- zone selections
- numeric values where applicable
- file loading exceptions

Invalid input must trigger a clear error message and a re-prompt.

9. Pseudocode Requirements

You must submit pseudocode for:

- all classes
- the entire program
- all major algorithms
- menu logic
- file loading logic
- statistical calculations

Pseudocode must follow CLUCC principles:

- Clear
- Logical
- Understandable
- Consistent
- Correct

10. Java Implementation Requirements

Your Java code must:

- compile and run in the Linux VDI command line environment on the lab machines
- use only standard Java libraries
- follow coding standards
- include meaningful comments
- handle exceptions gracefully
- write to a log file
- use arrays (not ArrayLists)

11. Deliverables

You must submit:

1. Pseudocode for the entire program
2. Java source code
3. UML Diagram of the system
4. Log file generated by your program
5. Test plan
6. List of known errors (if any)
7. READ ME file: covering dependencies and a how to use it

8. Reflection on your design choices
9. Signed Student Declaration

12. Reflection Requirements

Your reflection should discuss:

- why you chose specific data types
- how you structured your classes
- how you approached the menu system
- challenges you encountered
- what you would improve with more time

13. Submission Details

- **Due Date & Time:** Sunday 17 May 2026 @ 23:59 (AWST)
- **Submission Platform:** via the Assignment submission point in Blackboard
- **File Formats and naming conventions:**
 - A single .zip folder containing all below listed files: studentNumber_assignment.zip
 - .pdf for pseudocode, test plan, UML diagram and reflection
 - studentNumber_pseudocode.pdf, studentNumber_testPlan.pdf, studentNumber_UML.pdf and studentNumber_reflection.pdf
 - .java files for code
 - .txt for log file and READ ME
 - studentNumber_logFile.txt, studentNumber_README.txt

14. Demonstration and Viva

You will be required to demonstrate you submitted code and complete a viva during your practical session in the last week of semester. Please read Unit Outline to understand the significance of this.

15. Academic Integrity

All work must be your own. Plagiarism or codesharing will be handled according to university policy.

Important things to note

This is to be an Object Oriented solution. The use of arrays of objects is expected.

*The use of ArrayLists, StringBuilder, Hash and HashMap related classes are **NOT** permitted. You must manually create and iterate over the arrays. You must only use the constructs that we have used within the unit. File Reading and Writing cannot use Scanner. If you are unsure, please speak with either your practical staff, local staff or the unit coordinator.*